

Assessment 3 Synchronization & Federated Learning - Chawin Luangtumrongjareon

System Components

- **Server (server.rs):**
 - Manages client registration, model initialization, and aggregation of model updates.
 - Hosts the global model (a linear model for MNIST) and coordinates training rounds.
 - Provides a command-line interface (CLI) for initiating training, testing the model, and retrieving model parameters.
 - Listens for TCP connections on port 50051 and handles client messages asynchronously.
- **Client (client.rs):**
 - Connects to the server, registers itself, and receives model parameters for local training.
 - Trains the model on a subset of the MNIST dataset (10,000 randomly selected samples) using stochastic gradient descent (SGD).
 - Sends updated model parameters (weights and biases) back to the server after training.
 - Listens for server commands (e.g., TRAIN, GET, TEST) and responds accordingly.
- **Model Library (lib.rs):**
 - Defines the LinearModel structure, which implements a simple linear layer for classifying MNIST digits (784 input features, 10 output classes).
 - Provides traits and methods for model initialization, forward pass, and accessing weights and biases.
 - Uses Candle for tensor operations and initialization (e.g., Kaiming normal initialization for weights, zero initialization for biases).

Assessment 3 Synchronization & Federated Learning - Chawin Luangtumrongjareon

Data Flow

The federated learning process follows these steps:

1. **Initialization:**
 - The server initializes the global model and loads the MNIST test dataset.
 - Clients connect to the server, register their IP addresses, and signal readiness.
2. **Training Rounds:**
 - The server selects a subset of ready clients (configurable number) and sends the current global model (weights and biases) to each client.
 - Clients train the model locally on their subset of the MNIST training data for a specified number of epochs using SGD.
 - Clients send their updated model parameters (weights and biases) back to the server.
3. **Aggregation:**
 - The server aggregates the client updates by computing the average of weights and biases across all participating clients.
 - The aggregated model becomes the new global model, which is used for the next round.
4. **Testing and Completion:**
 - The server periodically tests the global model on the MNIST test dataset to evaluate accuracy.
 - After all training rounds are complete, the server notifies all clients and shuts down gracefully.

Key Features

- Asynchronous, non-blocking I/O using Tokio for scalability.
 - Privacy-preserving federated learning (no raw data shared).
 - Error handling and robustness against client disconnections.
 - Configurable training parameters (number of clients, rounds, epochs).
-

Assessment 3 Synchronization & Federated Learning - Chawin Luangtumrongjareon

Implementation Details and Clarifications

The implementation is already provided in the three Rust files (server.rs, client.rs, and lib.rs). Below, I'll highlight key aspects of the implementation, explain how it works, and suggest potential improvements or clarifications.

1. Server Implementation (server.rs)

Key Functions:

- **Server:** Struct that holds state (clients, model, dataset) and methods for registration, initialization, aggregation, and training.
- **aggregate_updates:** Averages client updates (weights and biases) to update the global model. It checks for correct dimensions and handles errors.
- **train:** Coordinates training rounds, selects random clients, and aggregates their updates. Uses mpsc (multi-producer, single-consumer channel) for collecting updates.
- **handle_client:** Processes client messages (REGISTER, READY, GET, TEST) asynchronously.
- **handle_commands:** Provides a CLI for server commands (INIT, TRAIN, GET, TEST).

Notes:

- The server uses `Arc<Mutex>` for thread-safe state management across async tasks.
- Model parameters are serialized/deserialized using `bincode` and encoded in `base64` for network transmission.
- The MNIST test accuracy is calculated by comparing predicted labels (`argmax` of logits) with true labels.

2. Client Implementation (client.rs)

Key Functions:

- **Client:** Struct that manages connection to the server, local model, and dataset subset.
- **join:** Establishes a connection to the server, registers the client, and sets up a listener for server commands.
- **train:** Performs local training using SGD on a subset of 10,000 MNIST samples for the specified epochs.
- **run_inner:** Listens for server messages (TRAIN, GET, TEST, COMPLETE) and responds accordingly.

Notes:

- Each client trains on a random subset of the MNIST training data, ensuring diversity in local updates.

Assessment 3 Synchronization & Federated Learning - Chawin Luangtumrongjareon

- The client uses TcpListener to accept commands from the server, allowing bidirectional communication.
- Model updates are sent back to the server in the same format as received (base64-encoded, serialized).

3. Model Library (lib.rs)

Key Functions:

- LinearModel: Implements a linear layer with 784 inputs and 10 outputs, using Candle's Linear module.
- Model trait: Defines methods for model initialization, forward pass, and parameter access.

Notes:

- The model is simple but effective for demonstrating federated learning on MNIST.
- Weights are initialized with Kaiming normal, and biases are initialized to zero, which is standard for neural networks.

4. Communication Protocol

The system uses TCP for communication between the server and clients, with messages formatted as strings separated by the '|' character. Example messages include:

- **Client to Server:**
 - REGISTER|<client_ip>: Registers the client with the server.
 - READY: Signals that the client is ready to participate in training.
 - UPDATE|<encoded_weights>|<encoded_biases>: Sends updated model parameters to the server.
- **Server to Client:**
 - TRAIN|<model_name>|<encoded_weights>|<encoded_biases>|<epochs>: Instructs the client to train the model for a specified number of epochs.
 - COMPLETE: Notifies the client that training is complete.
 - MODEL|<encoded_weights>|<encoded_biases>|<status>: Returns the current model parameters and status.
 - ACCURACY|<accuracy>: Returns the test accuracy of the model.

Data (e.g., weights and biases) is serialized using bincode, encoded in base64, and transmitted as part of the message to ensure compatibility and efficiency.

Assessment 3 Synchronization & Federated Learning - Chawin Luangtumrongjareon

Key Design Decisions

1. Asynchronous Networking with Tokio:

- Tokio is used to handle asynchronous TCP connections, allowing the server to manage multiple clients concurrently without blocking.
- The server uses a TcpListener to accept connections, and each client connection is handled in a separate async task.
- This design ensures scalability and responsiveness, even with a large number of clients.

2. Error Handling and Robustness:

- The system includes comprehensive error handling using Result and anyhow for propagating errors across async boundaries.
- Clients and the server can handle disconnections gracefully, removing unregistered or disconnected clients from tracking.
- The server validates model updates (e.g., checking dimensions) to prevent crashes or incorrect aggregation.

3. Configuration and Flexibility:

- Training parameters (number of clients, rounds, epochs) are configurable via the server's CLI.
- The system supports dynamic client participation, allowing clients to join or leave at any time during the process.